Assignment #2

Yamakawa's Fuzzy Controller



INSTRUCTIONS



DOCUMENTATION GUIDE



MEMBERSHIP FUNCTION VIEWER

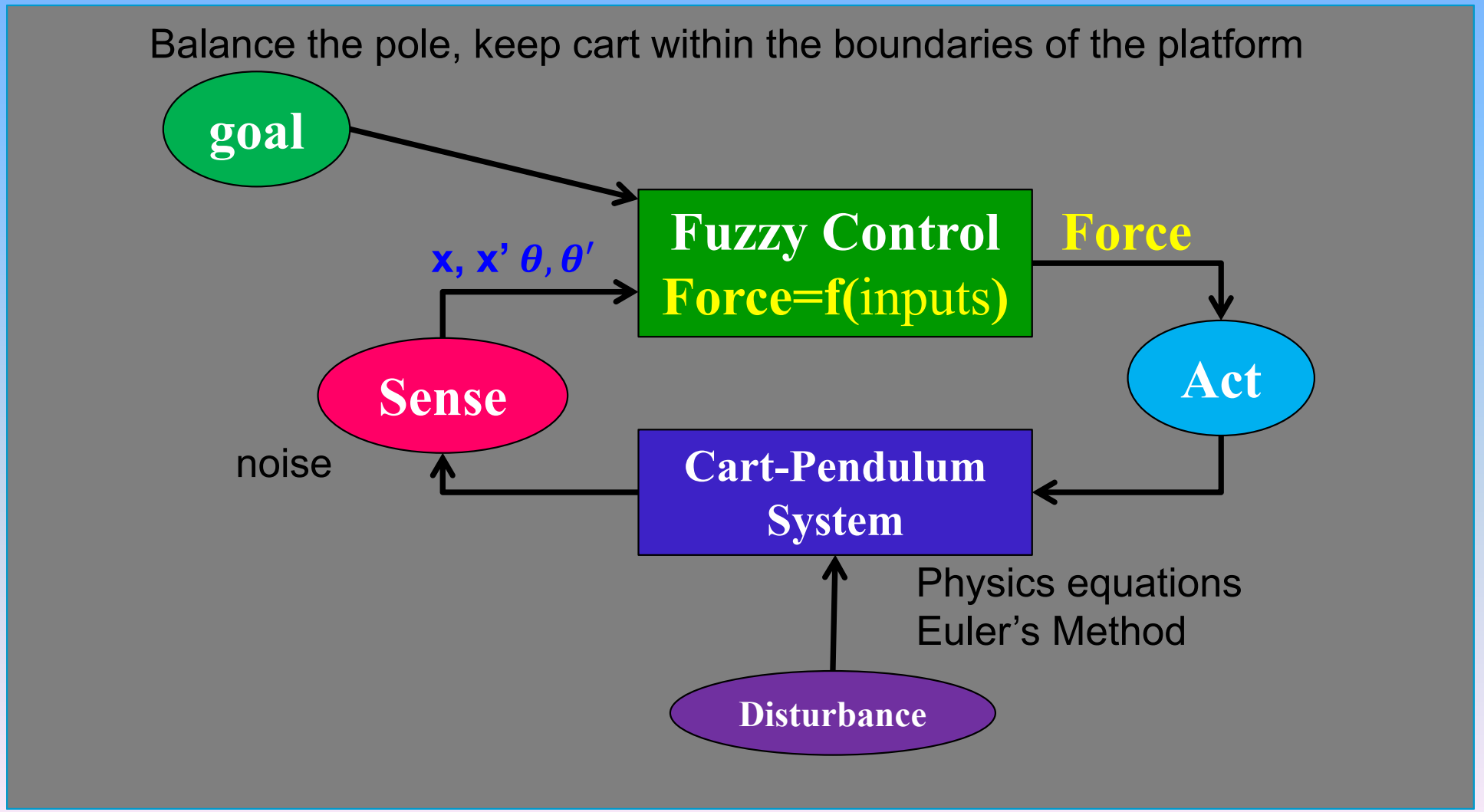


CHECKLIST

n.h.reyes@massey.ac.nz

Feedback Control/Closed Loop Control

- Closed-loop control allows for **uncertainty** in the model as well as **noise** and **disturbances** in the system under control



Solve the tutorial first, before studying this lecture.

Fuzzy Logic Tutorial

2 Inputs, 9 rules

Fuzzy Controller

Start-up codes

n.h.reyes@massey.ac.nz

SEARCH

Start-up Codes: makefile

The start-up system is comprised of multiple files.

Executable filename

```
main.exe       :   main.o transform.o sprites.o fuzzylogic.o graphics.o
    g++ -O2 -std=c++11  -Wl,-s -o main.exe main.o transform.o sprites.o fuzzylogic.o graphics.o -l gdi32 -static-libgcc -static-libstdc++

main.o      :   main.cpp graphics.h transform.h sprites.h fuzzylogic.h
    g++ -O2  -std=c++11  -c -fpermissive -fconserve-space  -Wno-write-strings main.cpp

transform.o     :    transform.cpp transform.h
    g++ -O2  -std=c++11  -c -fpermissive -fconserve-space  -Wno-write-strings transform.cpp

sprites.o       :    sprites.cpp sprites.h graphics.h transform.h
    g++ -O2  -std=c++11  -c -fpermissive -fconserve-space  -Wno-write-strings sprites.cpp

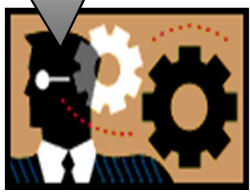
fuzzylogic.o    :    fuzzylogic.cpp fuzzylogic.h
    g++ -O2  -std=c++11  -c -fpermissive -fconserve-space  -Wno-write-strings fuzzylogic.cpp

graphics.o      :    graphics.cpp graphics.h
    g++ -O2  -std=c++11  -c -fpermissive -fconserve-space -Wno-write-strings  graphics.cpp

clean:
    del *.o
    del *.exe
```

Indented statements must be preceded by a **tab**.

The program requires that you "**build**" it using a **makefile**.



back

SEARCH

Start-up Codes: makefile

```
main.exe        :   main.o transform.o sprites.o fuzzylogic.o graphics.o
    g++ -O2 -std=c++11  -Wl,-s -o main.exe main.o transform.o sprites.o fuzzylogic.o graphics.o -l gdi32 -static-libgcc -static-libstdc++

main.o      :   main.cpp graphics.h transform.h sprites.h fuzzylogic.h
    g++ -O2  -std=c++11  -c -fpermissive -fconserve-space  -Wno-write-strings main.cpp

transform.o     :    transform.cpp transform.h
    g++ -O2  -std=c++11  -c -fpermissive -fconserve-space  -Wno-write-strings transform.cpp

sprites.o       :    sprites.cpp sprites.h graphics.h transform.h
    g++ -O2  -std=c++11  -c -fpermissive -fconserve-space  -Wno-write-strings sprites.cpp

fuzzylogic.o    :    fuzzylogic.cpp fuzzylogic.h
    g++ -O2  -std=c++11  -c -fpermissive -fconserve-space  -Wno-write-strings fuzzylogic.cpp

graphics.o      :    graphics.cpp graphics.h
    g++ -O2  -std=c++11  -c -fpermissive -fconserve-space -Wno-write-strings  graphics.cpp

clean:
    del *.o
    del *.exe
```

The program uses a simple graphics library (**graphics.cpp and graphics.h**) and links to the built-in graphics device interface (**gdi32**) of the Windows operating system. Because of this, the program only compiles in the Windows environment.



Fuzzy Logic Engine

C/C++ version

Neural Network and Fuzzy Logic Applications in C/C++ (Wiley Professional Computing) by Stephen Welstead

Fuzzy Inference System

float inputs[MAX_NO_OF_INPUTS]

fuzzy_system_rec fz

crisp_output = fuzzy_system(inputs, fz);

A function that can execute a complete fuzzy inference system.

We would like a function that feeds on an array of inputs and a complete specification of a fuzzy system, and in turn returns the final crisp output.



Enumerated data types – for easy indexing

fuzzylogic.h

```
// Indices
////////////////////////////////////////////////////////////////////////////////
typedef   enum {in_angle, in_distance};  //input indices

//indices of fuzzy sets for angle input
typedef   enum {in_negatively_small, in_negatively_medium, in_negatively_large, in_zero,
in_positively_small, in_positively_medium, in

//indices of fuzzy sets for distance input
typedef   enum {in_near, in_medium, in_f

//indices for output steering angle
typedef   enum {out_ze_turn, out_negativ
out_negatively_sharp_turn, out_negativel
out_positively_very_mild_turn, out_positi
out_positively_very_sharp_turn
};

//indices for output speed
typedef   enum {out_very_slow, out_slow, out_medium, out_fast, out_very_fast,
                out_wicked_fast};
```

Define **enum** variables for the linguistic terms.

We would like to refer to the different fuzzy sets and input parameters using descriptive names (e.g. {in_angle, in_position}, {in_negatively_small, in_negatively_large, etc.}), instead of numbers whenever we use them.

Add linguistic terms tailored to the problem domain



Trapezoidal Membership Function

fuzzylogic.h

```
//Trapezoidal membership function types
typedef   enum { regular_trapezoid, left_trapezoid, right_trapezoid }  trapz_type;


// Trapezoidal Fuzzy Set ////////////////////

struct  trapezoid {

   trapz_type    tp;  //type of trapezo

   float  a, b, c, d, l_slope, r_slope;

};
```

Define a structure for holding the properties of the membership functions to use.

Here we have a structure implementing the trapezoidal membership function.



Fuzzy Rule

fuzzylogic.h

```
// Fuzzy Rule ////////////////////////////////////////////////////////////

typedef   struct {

   short     inp_index[MAX_NO_OF_INPUTS],  //input index
             inp_fuzzy_set[MAX_NO_OF_INPUTS], //input fuzzy set index
             out_fuzzy_set;  //output index

} rule;
```

Define a structure for holding the components of a fuzzy rule.

Note that we are only storing the indices of the components of a rule.



inp_index[0]: **in_angle**

inp_index[1]: **in_distance**

out_fuzzy_set

If (**angle** is **small**) AND (**distance** is **VERY_FAR**) Then output is **VERY_FAST**.

inp_fuzzy_set[0]: **in_small**

inp_fuzzy_set[1]: **in_very_far**

Fuzzy System

fuzzylogic.h

```
// The complete Fuzzy System /////

typedef  struct fuzzy_system_rec {

    bool allocated;

    trapezoid   inp_mem_fns [MAX_NO_OF_INPUTS] [MAX_NO_OF_INP_REGIONS];

    rule*  rules; //note that we need to allocate memory for the rules

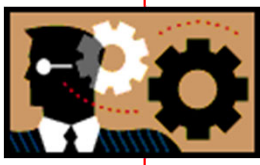
    int       no_of_inputs,          //number of inputs
              no_of_inp_regions,     //number of fuzzy sets associated with each input
              no_of_rules,           //number of rules
              no_of_outputs;         //number of fuzzy outputs

    float  output_values[MAX_NO_OF_OUTPUT_VALUES]; //the values of the outputs

};
```

Define a structure for holding the complete description of a **fuzzy inference system**.

Here we have a structure implementing a Zero-Order Sugeno Fuzzy inference system.



Defining the Fuzzy Rule

If (**angle** is small) **AND** (**distance** is VERY_FAR) **Then** output is **VERY_FAST**.

```
fuzzy_system_rec *fl;
//…allocate memory
//…initialise fuzzy parameters

    fl->rules[0].inp_index[0] = in_angle;
    fl->rules[0].inp_index[1] = in_distanc

    fl->rules[0].inp_fuzzy_set[0] = in_sm
    fl->rules[0].inp_fuzzy_set[1] = in_ver
    fl->rules[0].out_fuzzy_set = out_very
```

How to customise a fuzzy rule?

Here is an example of how we can define a rule with two inputs.

```
// Fuzzy Rule ////////////////////////////////////
typedef   struct {
  short     inp_index[MAX_NO_OF_INPUTS],
            inp_fuzzy_set[MAX_NO_OF_INPUTS],
            out_fuzzy_set;  //output index
} rule;
```

SMALL	Med Speed	Fast Sp	ry Fast
MEDIUM	Slow Speed	Med Speed	Fa
LARGE	Very Slow	Slow Speed	Sl



Defining the Fuzzy Rule

Main.cpp

inp_index[0]: **in_angle** inp_index[1]: **in_distance** **out_fuzzy_set**

If (**angle** is small) **AND** (**distance** is VERY_FAR) **Then** output is **VERY_FAST**.

```
fuzzy_system_rec *fl;
//…allocate memory
//…initialise fuzzy parameters

    fl->rules[0].inp_index[0] = in_angle;
    fl->rules[0].inp_index[1] = in_distance;

    fl->rules[0].inp_fuzzy_set[0] = in_small;
    fl->rules[0].inp_fuzzy_set[1] = in_very_far;
    fl->rules[0].out_fuzzy_set = out_very_fast;
```

We are only storing the indices in the rule.

```
// Fuzzy Rule ////////////////////////////////////////////
typedef   struct {
  short     inp_index[MAX_NO_OF_INPUTS],
            inp_fuzzy_set[MAX_NO_OF_INPUTS],
            out_fuzzy_set;  //output index
} rule;
```

FAMM

	NEAR	*FAR*	*VERY FAR*
SMALL	Med Speed	Fast Speed	Very Fast
MEDIUM	Slow Speed	Med Speed	Fast Speed
LARGE	Very Slow	Slow Speed	Slow Speed

Membership Function

If (**angle** is small) AND (**distance** is VERY_FAR) Then output is **VERY_FAST**.

fl->inp_mem_fns[variable_index][fuzzy_set]

```
fl->rules[i].inp_index[0] = in_angle;
fl->rules[i].inp_index[1] = in_distance

fl->rules[0].inp_fuzzy_set[0] = in_sma
fl->rules[0].inp_fuzzy_set[1] = in_very
fl->rules[0].out_fuzzy_set = out_very
```

How to specify which **membership function** we want to use?

Using the index of the rule input, and index of the fuzzy set, we can easily identify the membership function.

SMALL	Med Speed	Fast Speed	Fast
MEDIUM	Slow Speed	Med Speed	Fa
LARGE	Very Slow	Slow Speed	Sl



Membership Function

inp_index[0]: **in_angle**

inp_index[1]: **in_distance**

out_fuzzy_set

If (**angle** is small) AND (**distance** is VERY_FAR) Then output is **VERY_FAST**.

fl->inp_mem_fns[variable_index][fuzzy_set]

```
fl->rules[i].inp_index[0] = in_angle;
fl->rules[i].inp_index[1] = in_distance;

fl->rules[0].inp_fuzzy_set[0] = in_small;
fl->rules[0].inp_fuzzy_set[1] = in_very_far;
fl->rules[0].out_fuzzy_set = out_very_fast;
```

	NEAR	*FAR*	*VERY FAR*
SMALL	Med Speed	Fast Speed	Very Fast
MEDIUM	Slow Speed	Med Speed	Fast Speed
LARGE	Very Slow	Slow Speed	Slow Speed

Membership Function data structure

Declaration:

trapezoid **inp_mem_fns** [MAX_NO_OF_INPUTS] [MAX_NO_OF_INP_REGIONS];

inp_mem_fns is a structure containing all the parameters of a membership function.

fl->inp_mem_fns[variable_index][fuzzy_set]

```
fl->rules[i].inp_index[0] = in_angle;
fl->rules[i].inp_index[1] = in_distance;

fl->rules[0].inp_fuzzy_set[0] = in_small;
fl->rules[0].inp_fuzzy_set[1] = in_very_far;
fl->rules[0].out_fuzzy_set = out_very_fast;
```

Membership Function Initialisation

inp_mem_fns is a structure containing all the parameters of a membership function.

The **init_trapz()** function is used to initialize **inp_mem_fns**.

- fl->**inp_mem_fns**[in_angle][in_small] **=**
  - **init_trapz**(14.0f, 20.0f, 0.0f, 0.0f, **left_trapezoid**);

```
fl->rules[i].inp_index[0] = in_angle;
fl->rules[i].inp_index[1] = in_distance;

fl->rules[0].inp_fuzzy_set[0] = in_small;
fl->rules[0].inp_fuzzy_set[1] = in_very_far;
fl->rules[0].out_fuzzy_set = out_very_fast;
```

Calculation of degree of Membership

inp_mem_fns is a structure containing all the parameters of a membership function.

The **trapz** function calculates the **actual degree of membership** of a given input value in a fuzzy set.

which input

which membership function

trapz(inputs[variable_index],fl.inp_mem_fns[variable_index][fuzzy_set])

```
fl->rules[i].inp_index[0] = in_angle;
fl->rules[i].inp_index[1] = in_distance;

fl->rules[0].inp_fuzzy_set[0] = in_small;
fl->rules[0].inp_fuzzy_set[1] = in_very_far;
fl->rules[0].out_fuzzy_set = out_very_fast;
```

Calculation of degree of Membership

fuzzylogic.cpp

which input

which membership function

trapz(inputs[variable_index],fl.inp_mem_fns[variable_index][fuzzy_set])

```
float trapz (float x, trapezoid trz) {
  switch (trz.tp) {

    case left_trapezoid: //opening to the left
                if (x <= trz.a)
                  return 1.0;
                if (x >= trz.b)
                  return 0.0;
                /* a < x < b */
                return trz.r_slope * (x - trz.b);


    case right_trapezoid: //opening to the right
                if (x <= trz.a)
                  return 0.0;
                if (x >= trz.b)
                  return 1.0;
                /* a < x < b */
                return trz.l_slope * (x - trz.a);
```

```
    case regular_trapezoid:
                if ((x <= trz.a) || (x >= trz.d))
                  return 0.0;
                if ((x >= trz.b) && (x <= trz.c))
                  return 1.0;
                if ((x >= trz.a) && (x <= trz.b))
                  return trz.l_slope * (x - trz.a);
                if ((x >= trz.c) && (x <= trz.d))
                  return  trz.r_slope * (x - trz.d);

                } /* End switch  */

  return 0.0;  /* should not get to this point */
} /* End function */
```

Fuzzy System – from fuzzification to defuzzification

File: **fuzzylogic.cpp**

```
float  fuzzy_system ( float  inputs[ ],  fuzzy_system_rec   fz) {
   int  i, j;
   short  variable_index, fuzzy_set;
   float  sum1 = 0.0f,  sum2 = 0.0f,  weight;
   float  m_values[MAX_NO_OF_INPUTS];

   for (i = 0; i < fz.no_of_rules; i++) {
      for (j = 0; j < fz.no_of_inputs; j++) {
                  variable_index = fz.rules[i].inp_index[j];
                  fuzzy_set = fz.rules[i].inp_fuzzy_set[j];
                  m_values[j] = trapz(inputs[variable_index], fz.inp_mem_fns[variable_index][fuzzy_set]);
       } /* end j  */

      weight = min_of (m_values, fz.no_of_inputs);
      sum1 += weight * fz.output_values[ fz.rules[i].out_fuzzy_set ];
      sum2 += weight;

   } /* end i  */

   if (fabs(sum2) < TOO_SMALL) {  // TOO_SMALL = 1e-6
      cout << "\r\nFLPRCS Error: sum2 in fuzzy_system is 0." << endl;
      //exit(1);
      return 0.0;
   }

   return (sum1/sum2);
}
```

which input

which fuzzy set

How to use the Fuzzy Logic Engine?

Fuzzy System Development

1. Declare a global variable for the fuzzy system.

File: **Main.cpp**

```
e.g.  fuzzy_system_rec  g_fuzzy_system;
```

Define the maximum number of Fuzzy Sets to be used.

File: **fuzzylogic.h**

```
#define  MAX_NO_OF_INP_REGIONS  7
```

e.g. 7 Maximum Number of Fuzzy Sets allowed

Fuzzy System Development

2. Initialise the fuzzy system.

File: **fuzzylogic.cpp**

```
void   initFuzzySystem (fuzzy_system_rec*  fl) {

   fl->no_of_inputs = 2;  //inputs are handled 2 at a time only
   fl->no_of_inp_regions = 7; //number of fuzzy sets per input
   fl->no_of_rules = 49*2;          //number of rules
//----
   fl->output_values [out_small_push_left] = -15f;   //-15 Newtons
   fl->output_values [out_large_push_left] = -100f; //-100 Newtons

   //… and so on…
   …
   fl->rules = new rule [fl->no_of_rules];  //allocate memory for the rules

   initFuzzyRules (fl); //initialise the rules; this is user-defined
   initMembershipFunctions(fl); //initialise the membership functions; this is user-
                                //defined
}
```

Fuzzy System Development

3. Initialise fuzzy rules.

File: **fuzzylogic.cpp**

```
void initFuzzyRules (fuzzy_system_rec*  fl) {
  int i;
  for (i = 0;i < fl->no_of_rules;i++) {
     //(*fl).rules[i].inp_index[0] = in_angle; //alternatively
     fl->rules[i].inp_index[0] = in_angle;
     fl->rules[i].inp_index[1] = in_angle_dot;
  }

  fl->rules[0].inp_fuzzy_set[0] = in_negatively_small;
  fl->rules[0].inp_fuzzy_set[1] = in_falling_to_left_fast;
  fl->rules[0].out_fuzzy_set = out_large_push_left;

// define the other remaining rules next
// and so on...

}
```

Rule #0

This is an example only. The inputs and rule parameters will depend on your system design.



Fuzzy System Development

3. Initialise fuzzy rules.

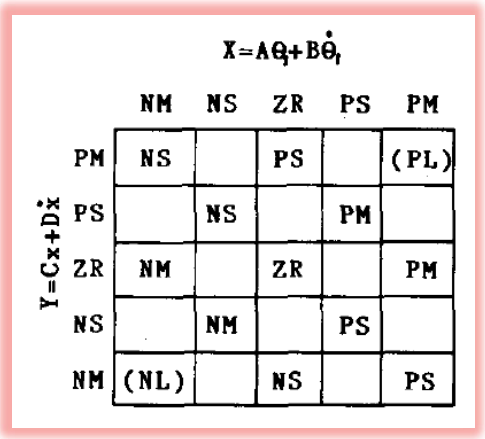
File: **fuzzylogic.cpp**

```
void initFuzzyRules (fuzzy_system_rec*  fl) {
  int i;
  for (i = 0;i < fl->no_of_rules;i++) {
     //(*fl).rules[i].inp_index[0] = in_X; //alternatively
     fl->rules[i].inp_index[0] = in_X;
     fl->rules[i].inp_index[1] = in_Y;
  }

  fl->rules[0].inp_fuzzy_set[0] = in_NM;
  fl->rules[0].inp_fuzzy_set[1] = in_PM;
  fl->rules[0].out_fuzzy_set = out_NS;

// define the other remaining rules next
// and so on...

}
```



Rule #0

Yamakawa's strategy



Fuzzy System Development

4. Initialise membership functions.

File: **fuzzylogic.cpp**

```
void   initMembershipFunctions ( fuzzy_system_rec*   fl ) {

  //angle
  fl->inp_mem_fns[in_angle][in_negatively_large] = init_trapz(14.0f,20.0f,0.0f,0.0f,
                                                                    left_trapezoid);
  fl->inp_mem_fns[in_angle][in_negatively_medium] = init_trapz(14.0f,20.0f,34.0f,40.0f,
                                                                    regular_trapezoid);
 //...
  fl->inp_mem_fns[in_angle][in_positively_large] = init_trapz (34.0f, 40.0f, 0.0f, 0.0f,
                                                                    right_trapezoid);


  //angular velocity
  //...
  //...
  //...
  //and so on...

}
```

which fuzzy set

Inverted Pendulum

4. Modify runInvertedPendulum ().

File: Main.cpp

```
void   runInvertedPendulum () {
  //...
   initFuzzySystem(&g_fuzzy_system);
   while((GetAsyncKeyState(VK_ESCAPE)) == 0 ) { //while ESC key is not pressed
      //...
       inputValues [ in_angle ] = prevState.angle;
       inputValues[ in_angle_dot ] = prevState.angle_dot;
       prevState.Force= fuzzy_system(inputValues, g_fuzzy_system);
       //...  Calculate new state of the world
       newState.angle_double_dot = calc_angular_acceleration(prevState);
       //and so on...
       cart.setX(newState.x);
       rod.setX(newState.x);
       rod.setAngle(newState.angle);
       cart.draw();   rod.draw();
}
    free_fuzzy_rules(&g_fuzzy_system);
   //and so on...
}
```

Specify the inputs and fuzzy system structure, then pass them to the fuzzy_system() function.



Skeleton of Simulation

void runInvertedPendulum()

```
initPendulumWorld();
initFuzzySystem(&g_fuzzy_system);

while((GetAsyncKeyState(VK_ESCAPE)) == 0 ) {

        setactivepage(page);
        cleardevice();

        senseEnvironment (inputs);

        F = fuzzy_system(inputs, g_fuzzy_system);

        updateWorld(newWorldState)
        drawCartAndRod(newWorldState)

        setvisualpage(page);
        toggle page  //switch to another page
}
```

Input and Output

INPUTS

1. Angle of the rod with respect to the vertical axis (in radians)
2. Angular velocity of the rod (in radians per second)
3. Cart position (in meters)
4. Cart velocity (in meters per second)

OUTPUT

OUTPUT

1. Force to apply (in Newtons)

World State

```
struct WorldStateType{
    void init(){
        x=0.0;
        x_dot=0.0;
        x_double_dot = 0.0;
        angle = 0.0;
        angle_dot = 0.0;
        angle_double_dot = 0.0;
        F = 0.0;
    }

    float x;
    float x_dot;
    float x_double_dot;
    float angle;
    float angle_dot;
    float angle_double_dot;
```

```
float const mb=0.1; // mass of broom
float const g=9.8; // pull of gravity
float const m=1.1; // mass of cart & broom
float const l=0.5; // length of broom from
//pivot point to centre of mass

float F;

};
```

WorldStateType **prevState, newState;**

Dynamics of the System

```
float calc_angular_acceleration( const WorldStateType& s){
    float a_double_dot=0.0;

    a_double_dot = (s.m * s.g * sin(s.angle) - (cos(s.angle)
                        * (s.F + ((s.mb) * s.l * s.angle_dot * s.angle_dot * sin(s.angle) ) ) )  )
                       / (  ((4/3)*s.m * s.l) - (s.mb * s.l * cos(s.angle) * cos(s.angle)));

    return a_double_dot;
}
```

```
float calc_horizontal_acceleration( const WorldStateType& s){
     float x_double_dot=0.0;

     x_double_dot = ( s.F + s.mb * s.l * (s.angle_dot * s.angle_dot)* sin(s.angle) -
                         s.angle_double_dot * cos(s.angle)   )/ s.m;

    return x_double_dot;
}
```

void runInvertedPendulum()

```
//*********************************************************
//Set the initial angle of the pole with respect to the vertical
prevState.angle = 8 * (3.14/180);  //initial angle  = 8 degrees

initFuzzySystem(&g_fuzzy_system);

while((GetAsyncKeyState(VK_ESCAPE)) == 0 ) {

 setactivepage(page);
 cleardevice();
 drawInvertedPendulumWorld();

 //retrieve inputs
 inputs[in_theta] = prevState.angle;
 inputs[in_theta_dot] = prevState.angle_dot;
 inputs[in_x] = prevState.x;
 inputs[in_x_dot] = prevState.x_dot;

 //1) Enable this only after your fuzzy system has been completed already.
 //Remember, you need to define the rules, membership function parameters and rule outputs.
 //prevState.F = fuzzy_system(inputs, g_fuzzy_system); //call the fuzzy controller

 externalForce=0.0;
 externalForce = getKey(); //manual operation

 if(externalForce != 0.0)
    prevState.F = externalForce;
//-------------------------------------------------------------------------
//continued…
```

Force to apply.

void runInvertedPendulum()

```
//-------------------------------------------------------------------------
 // ************************************************************************
 // BEGIN - DYNAMICS OF THE SYSTEM
 //Calculate the new state of the world
 newState.angle_double_dot = calc_angular_acceleration(prevState);
 newState.angle_dot = prevState.angle_dot + (h * newState.angle_double_dot);
 newState.angle = prevState.angle + (h * newState.angle_dot);
 newState.F = prevState.F;
 newState.x_double_dot = calc_horizontal_acceleration(prevState);
 newState.x_dot = prevState.x_dot + (h * newState.x_double_dot);
 newState.x = prevState.x + (h * newState.x_dot);
 prevState.x = newState.x;
 prevState.angle = newState.angle;
 prevState.x_dot = newState.x_dot;
 prevState.angle_dot = newState.angle_dot;
 prevState.angle_double_dot = newState.angle_double_dot;
 prevState.x_double_dot = newState.x_double_dot;
 //-------------------------
 cart.setX(newState.x);
 rod.setX(newState.x);
 rod.setAngle(newState.angle);
 cart.draw();
 rod.draw();
 // END - DYNAMICS OF THE SYSTEM
 // ************************************************************************
 //-------------------------------------------------------------------------
 displayInfo(newState);
 setvisualpage(page);
 page = !page;  //switch to another page
}
/2) Enable this only after your fuzzy system has been completed already.
//free_fuzzy_rules(&g_fuzzy_system);
```

Physics equations and Euler's method (please do not modify these statements)

Control surface generation

int main()

```
int main() {

   int graphDriver = 0,graphMode = 0;

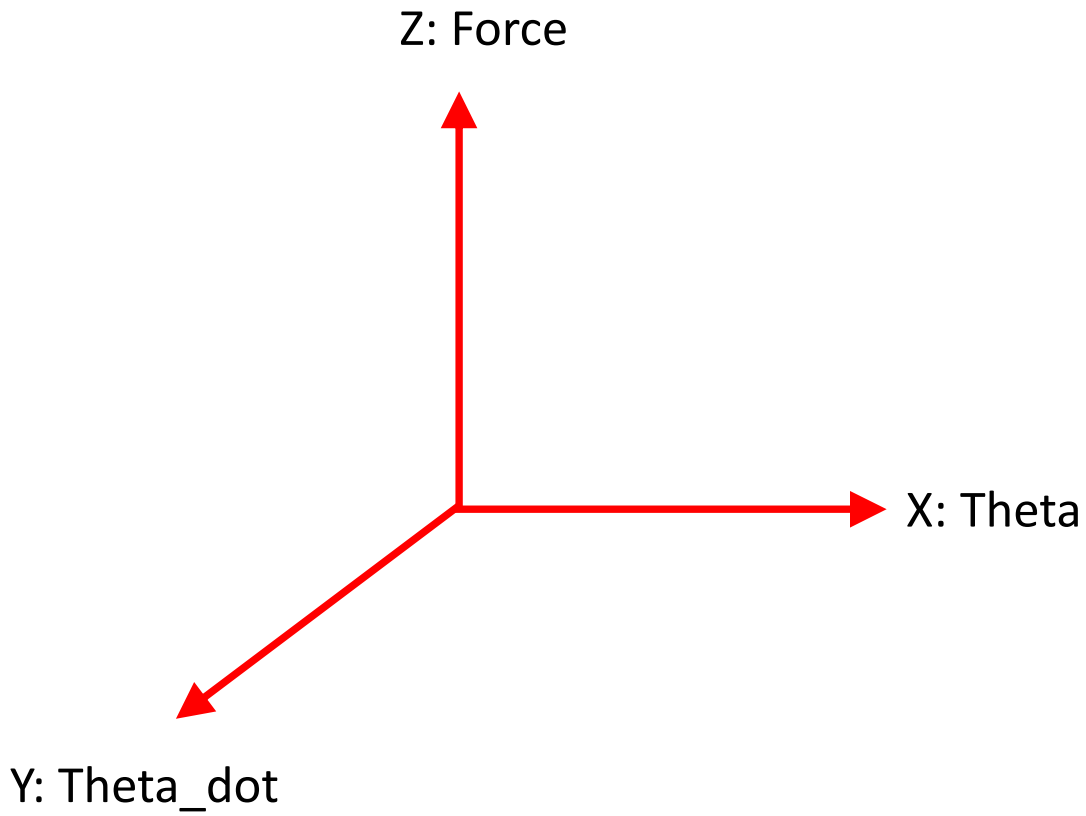
   initgraph(&graphDriver, &graphMode, "", 1280, 1024); // Start Window
   clearDataSet();
   try{
         runInvertedPendulum();

         //3) Enable this only after your fuzzy system has been completed already.
         //generateControlSurface_Angle_vs_Angle_Dot();

         //4) Enable this only after your fuzzy system has been completed already.
         //saveDataToFile("data_angle_vs_angle_dot.txt");

   }
   catch(...){
    cout << "Exception caught!\n";
   }
return 0;
}
```

Control surface generation



X: Theta [-40 deg., 40 deg] = [-0.7 rad., 0.7 rad.]

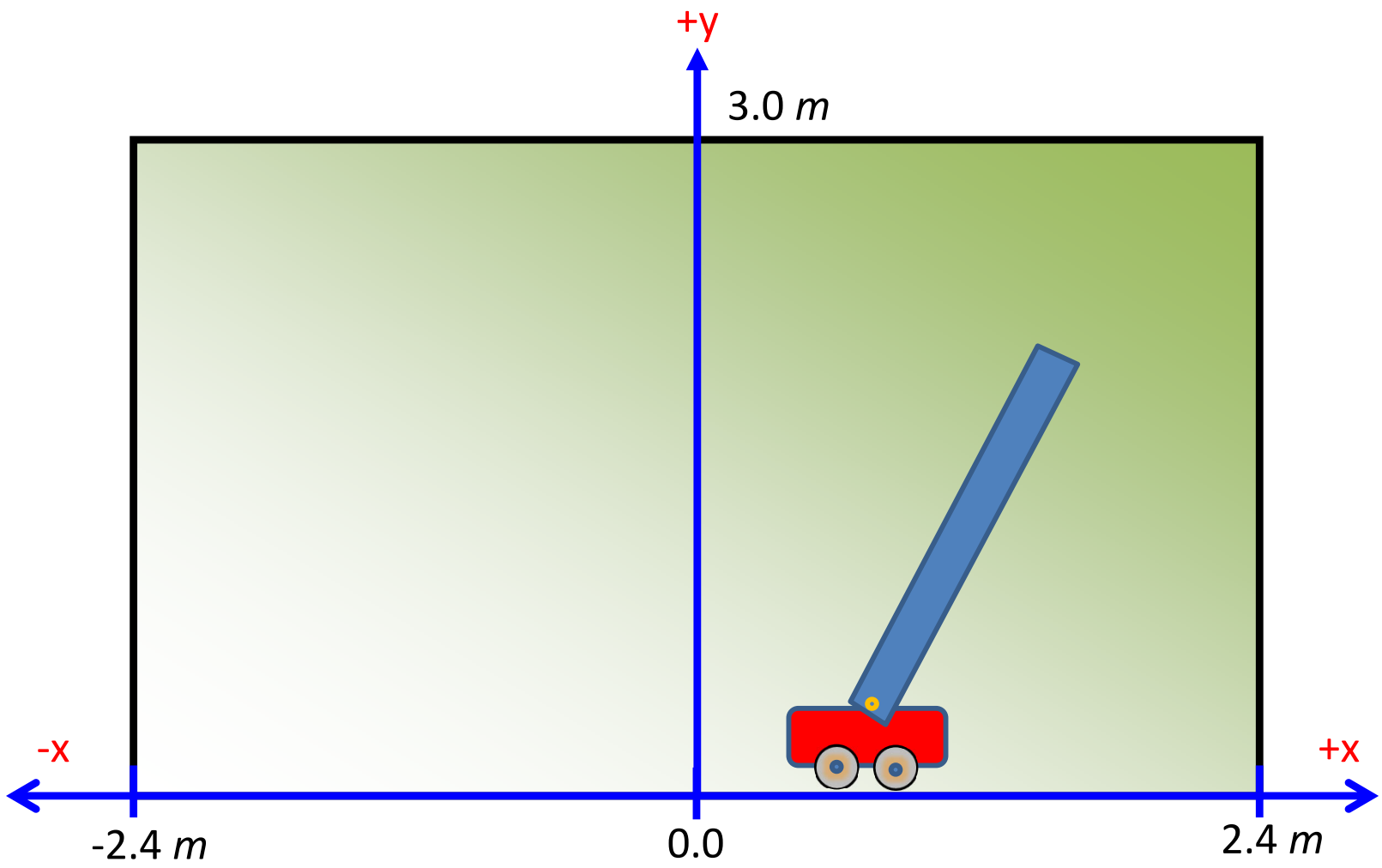
Y: Theta_dot = [-3 rad./sec, 3 rad./sec]

World Coordinate to Device Coordinate Transformation

for defining the system of coordinates, scaling and zooming-in or out.

Transformation Equations

WORLD SYSTEM OF COORDINATES



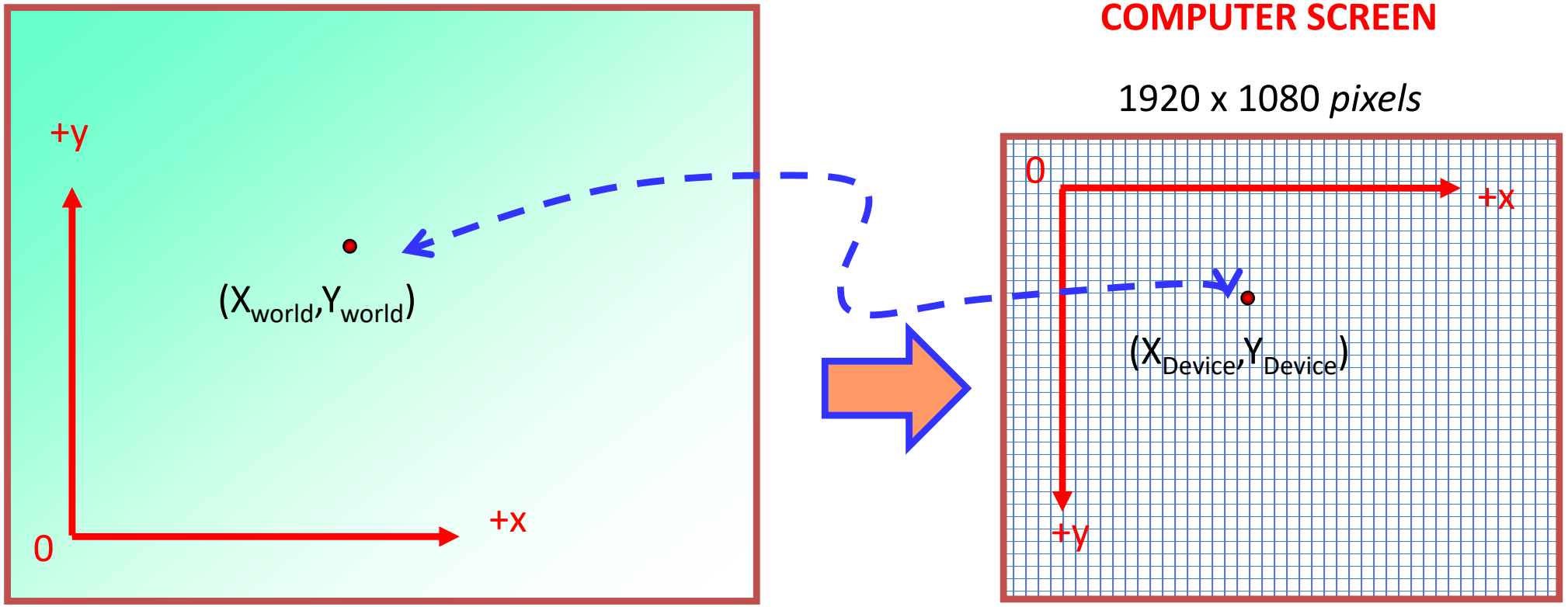
World System of Coordinates

(X_{world}, Y_{world})

Transformation Equations

WORLD-to-DEVICE COORDINATES

100,000,000 *m* x 500,000 *m*



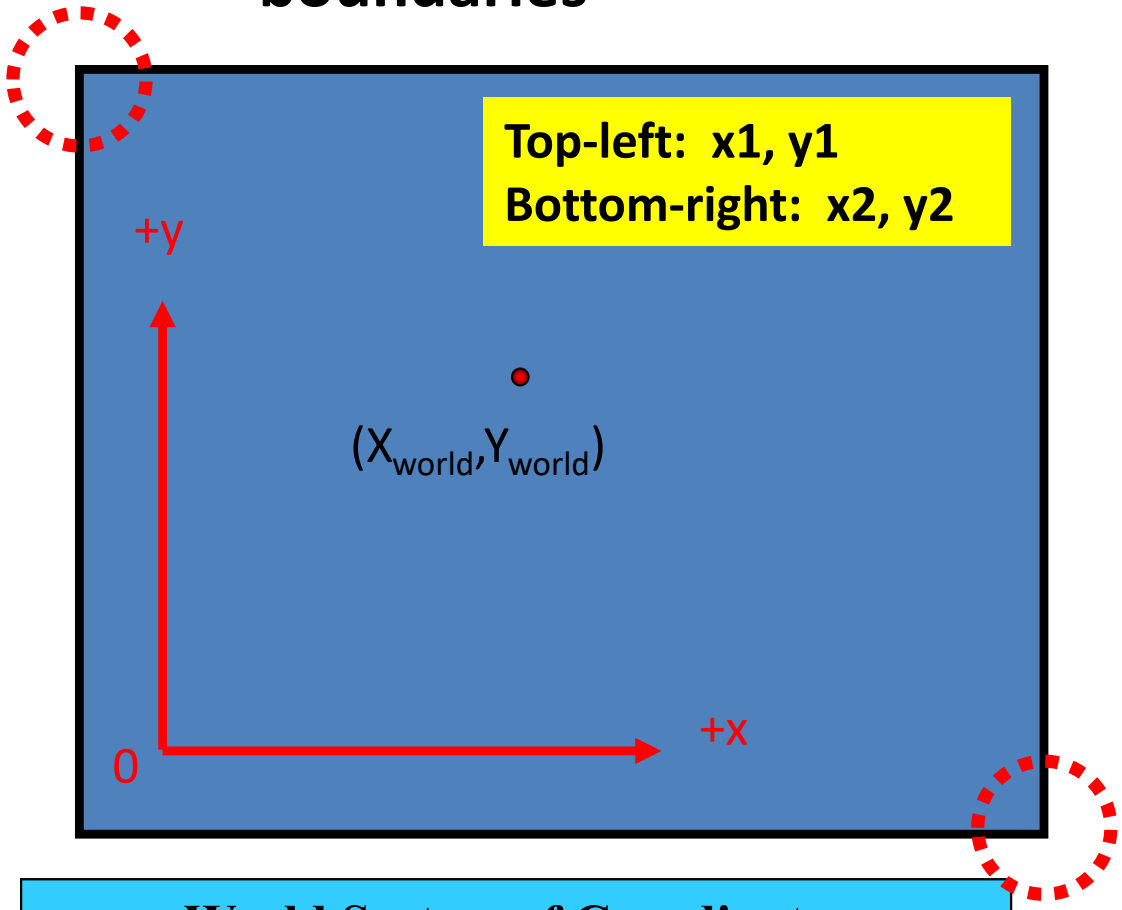
World System of Coordinates

Device System of Coordinates

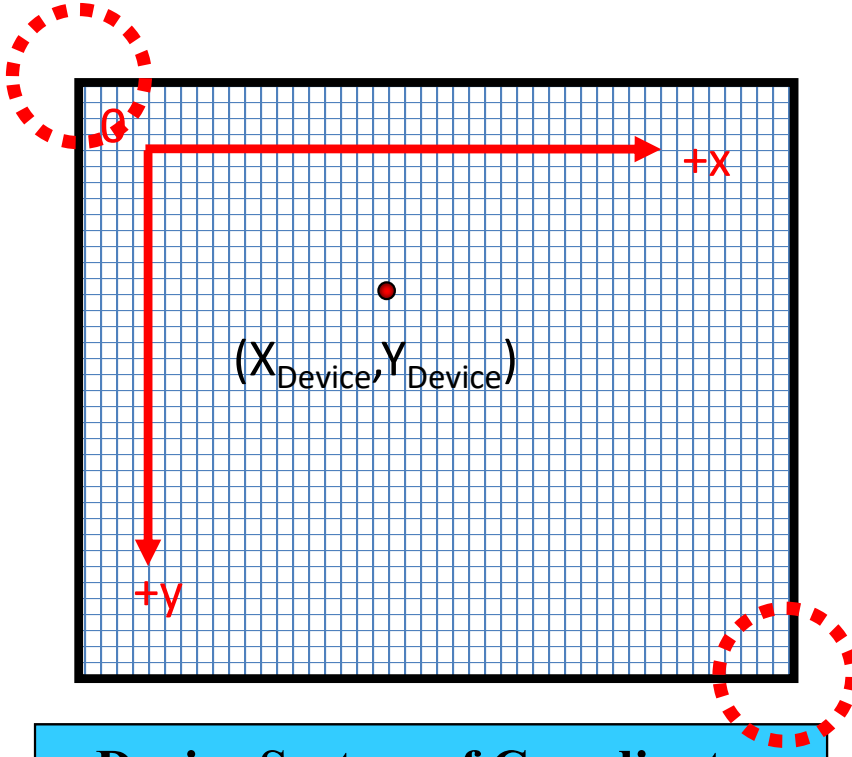
World Boundaries

SETTING THE BOUNDARIES

Use the upper-left and bottom-right coordinates to set the boundaries



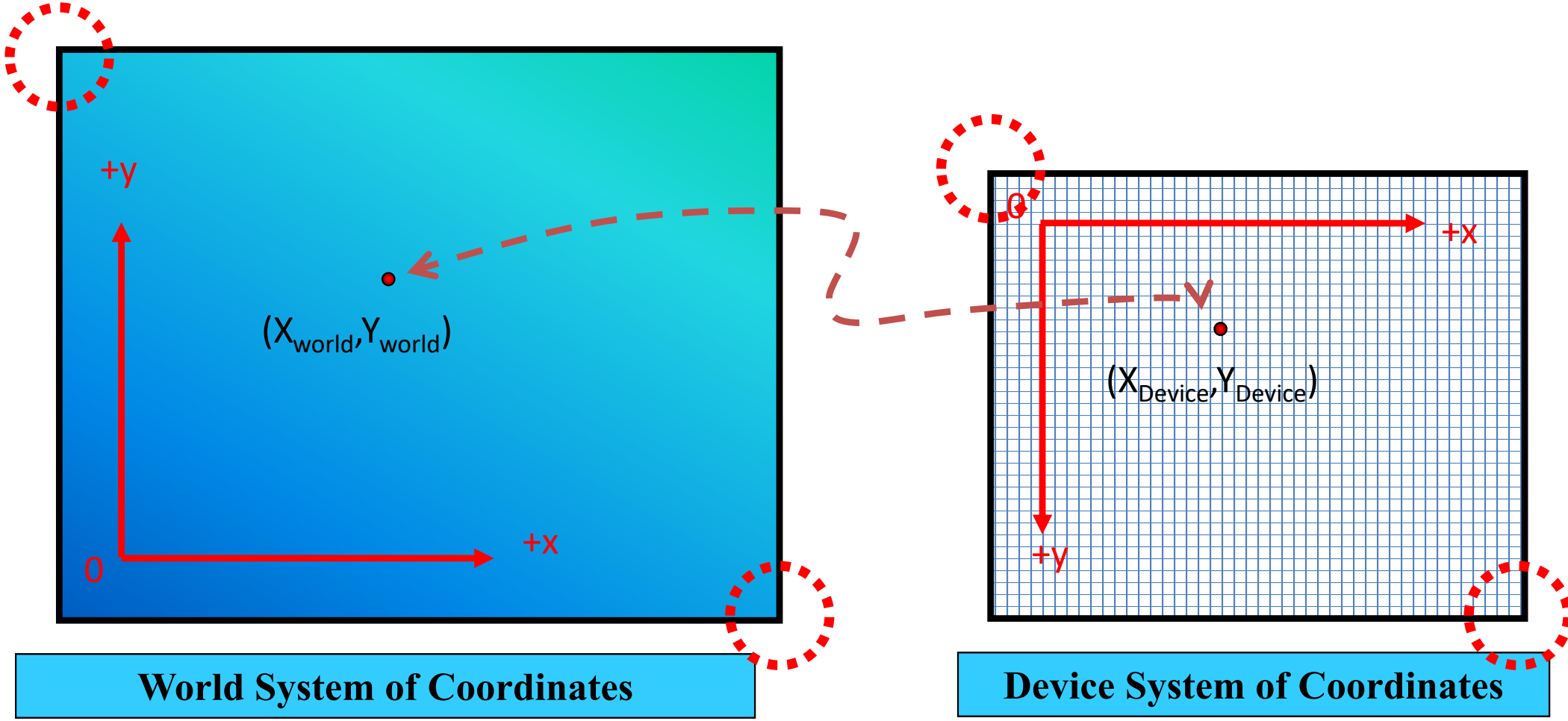
World System of Coordinates



Device System of Coordinates

Transformation

Given a world coordinate, what's the equivalent device coordinate?



World System of Coordinates

Device System of Coordinates

World-to-Device Coordinates

- **TRANSFORMATION EQUATIONS**

$$Xslope = \frac{DeviceBound_{X_2} - DeviceBound_{X_1}}{WorldBound_{X_2} - WorldBound_{X_1}}$$

$$\text{x_intercept} = DeviceBound_{X_1} - Xslope * WorldBound_{X_1}$$

$$Device_X = Xslope * \text{World}_X + \text{x_intercept}$$

World x-coordinate

For calculating Device_y, replace all **x**'s with **y**

Demonstration

Let’s see a sample solution…

How do we generate the 3D control surface?

Some Extras...

Randomisation

Element of Surprise

srand()

Seed for random-number generation

Seed the random-number generator with current time so that the numbers will be different every time we run.

```
srand( (unsigned)time( NULL ) );

/* Display 10 numbers. */
for( i = 0;   i < 10;i++ )
   printf( "  %6d\n", rand() );
```

Element of Surprise

- **rand()**

  generates a pseudorandom number - returns int

```
int  randomVal(int min, int max)
{
        return (min + (rand() % ((max-min)+1)  ));
}
```

rand() returns a pseudo-random integral number in the range (0 to RAND_MAX-1)

Element of Surprise

- rand()

```
float randomVal(float min, float max)
{
        float r;

        r = (float)rand()/RAND_MAX;
        r = min + (r*(max-min));

        return r;
}
```

rand() returns a pseudo-random integral number in the range (0 to RAND_MAX)-1

Time elapsed, wait…

- clock()

```
void wait ( int seconds )
{
  clock_t  endwait;

  endwait = clock () + seconds * CLOCKS_PER_SEC ;
  while (clock() < endwait) {}
}
```

```
clock_t  startTime, elapsedTime;

startTime = clock();
…
...
elapsedTime = (clock() - startTime) / CLOCKS_PER_SEC;
```

Keyboard Handling

Function keys, arrow keys, Control and Shift keys, combination of keys

Keyboard Handling

GetAsyncKeyState

The GetAsyncKeyState function determines whether a key is up or down at the time the function is called, and whether the key was pressed after a previous call to GetAsyncKeyState.

```
SHORT  GetAsyncKeyState( int vKey );    // vKey - virtual-key code
```

Virtual-key code

e.g.

vk_shift

vk_control

To find other pre-defined constants:
Using ***google***, type the following keywords: **msdn vk_shift**

http://msdn.microsoft.com/en-us/library/ms645540(VS.85).aspx

Keyboard Handling

GetAsyncKeyState

```
void moveSprite()
{

  if(GetAsyncKeyState(VK_UP) < 0) {
   SpriteY = SpriteY + 2;  //up
   outtext("UP");
  }

  if(GetAsyncKeyState(VK_DOWN) < 0) {
    SpriteY = SpriteY - 2;  //down
    outtext("DOWN");
    ....
```

To find other pre-defined constants:

Using google, type the following keywords: **msdn virtual key codes**

http://msdn.microsoft.com/en-us/library/ms645540(VS.85).aspx

Keyboard Handling

GetAsyncKeyState

```
//general key press event
if(GetAsyncKeyState(VK_SPACE) < 0) {
...
}


//key down event
if((GetAsyncKeyState(VK_SPACE) & 0X0001) ==0X0001){
...
}

//key up event
if((GetAsyncKeyState(VK_SPACE) & 0X8000) ==0X8000){
...
}
```

Keyboard Handling

Monitoring the **Control** and **Shift keys**:

```
bool ControlFlag, ShiftFlag;

if(GetAsyncKeyState(VK_CONTROL)<0)
 {
     ControlFlag =! ControlFlag;
 }

if(GetAsyncKeyState(VK_SHIFT)<0)
 {
     ShiftFlag =! ShiftFlag;
 }
```

For the Tank to Jump to the Right: Control + Shift + Right Arrow key

For the Tank to Jump to the Left: Control + Shift + Left Arrow key

Keyboard Handling

Possible approach in monitoring key combinations :

```
if(GetAsyncKeyState(VK_RIGHT)<0)
  {
       XDir=RIGHT;
       if(ShiftFlag)
        {
          outtext("SHIFT + RIGHT");
          ShiftFlag=!ShiftFlag;
        }

       if(ControlFlag)
        {
          outtext("CTRL + RIGHT");
          if (TankX < getmaxx()-W) TankX += 2;
          Angle=Angle-5;
          RaiseWheelFlag=TRUE;
          ControlFlag=!ControlFlag;
        }

       ...
```

End of Presentation

Let me answer some of your questions…



*